Exploratory Data Analysis and Visualization of Student Academic Success

Student: Adharsh R | **Roll No.:** 24AD002
Course: U21ADP05 – Exploratory Data Analysis and Visualization

Project objective

Explore student demographic, academic, and socioeconomic data; identify patterns associated with graduation, dropout, or continued enrolment; and train a Multilayer Perceptron (MLP) classifier. The dataset is obtained from the UCI Machine Learning Repository.

> Run every cell from top to bottom in Google Colab. The dataset is downloaded in the notebook. Model scores and plots must be taken from the actual run, not guessed.

```
# 1. Import libraries
import os, zipfile, urllib.request, glob, warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler,
LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.metrics import (classification_report, confusion_matrix,
accuracy_score,
                             ConfusionMatrixDisplay, roc_curve, auc)
from sklearn.neural_network import MLPClassifier

RANDOM_STATE = 42
sns.set_theme(style="whitegrid")
```

1. Dataset selection and loading

The UCI *Predict Students' Dropout and Academic Success* dataset contains student-related attributes and a three-class target: Dropout, Enrolled, and Graduate. The source dataset contains more than the required 15 fields.

```
# Download and extract the official UCI dataset
url =
"https://archive.ics.uci.edu/static/public/697/predict+students+dropou
```

```
t+and+academic+success.zip"
zip_path = "student_success.zip"
if not os.path.exists(zip_path):
    urllib.request.urlretrieve(url, zip_path)

extract_dir = "student_success_data"
os.makedirs(extract_dir, exist_ok=True)
with zipfile.ZipFile(zip_path, "r") as z:
    z.extractall(extract_dir)

csv_files = glob.glob(os.path.join(extract_dir, "**", "*.csv"),
recursive=True)
print("CSV files found:", csv_files)
if not csv_files:
    raise FileNotFoundError("No CSV file found after extraction. Check
the dataset URL or download the CSV manually.")
data_path = next((p for p in csv_files if "data" in
os.path.basename(p).lower()), csv_files[0])
df = pd.read_csv(data_path, sep=";")
print("Dataset shape:", df.shape)
display(df.head())
print("Number of fields:", df.shape[1])
```

```
CSV files found: ['student_success_data/data.csv']
Dataset shape: (4424, 37)
```

{"type":"dataframe"}

```
Number of fields: 37
```

2. Data understanding and preprocessing

Inspect field types, missing values, duplicate records, target balance, and summary statistics. Remove exact duplicate rows if present. Numeric missing values are imputed with the median; categorical missing values are imputed with the most frequent value. Numeric features are standardized and categorical features are one-hot encoded within a pipeline to reduce data leakage.

```
print("Shape:", df.shape)
print("\nData types:\n", df.dtypes.value_counts())
print("\nMissing values (top 20):\n",
df.isna().sum().sort_values(ascending=False).head(20))
print("\nDuplicate rows:", df.duplicated().sum())
print("\nDescriptive statistics:")
display(df.describe(include="all").T.head(25))

target_col = "Target"
if target_col not in df.columns:
    raise ValueError(f"Expected target column '{target_col}'.
```

```
Available columns: {df.columns.tolist()}")
df = df.drop_duplicates().copy()
df[target_col] = df[target_col].astype(str)
print("\nTarget distribution:\n", df[target_col].value_counts())

Shape: (4424, 37)

Data types:
 int64      29
float64     7
object      1
Name: count, dtype: int64

Missing values (top 20):
 Marital status                    0
Application mode                   0
Application order                  0
Course                             0
Daytime/evening attendance\t       0
Previous qualification             0
Previous qualification (grade)     0
Nacionality                        0
Mother's qualification             0
Father's qualification             0
Mother's occupation                0
Father's occupation                0
Admission grade                    0
Displaced                          0
Educational special needs          0
Debtor                             0
Tuition fees up to date            0
Gender                             0
Scholarship holder                 0
Age at enrollment                  0
dtype: int64

Duplicate rows: 0

Descriptive statistics:

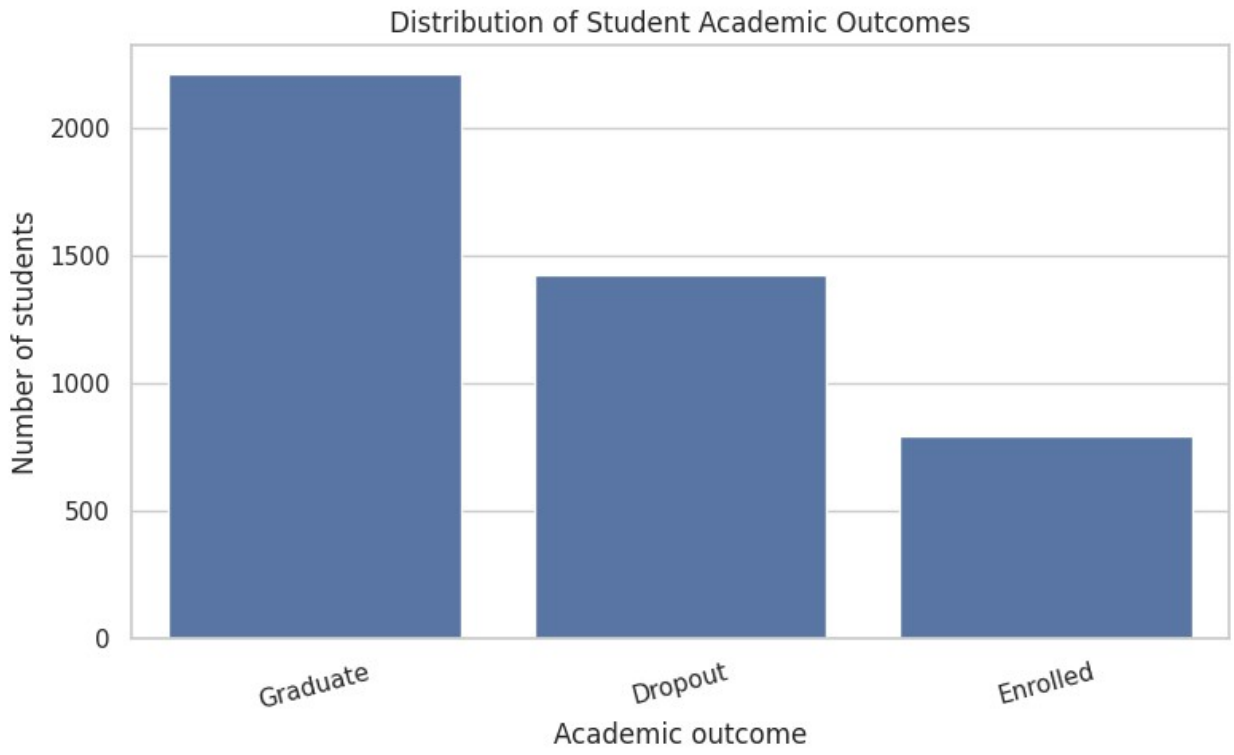
{"repr_error":"Out of range float values are not JSON compliant:
nan","type":"dataframe"}


Target distribution:
 Target
Graduate    2209
Dropout     1421
Enrolled     794
Name: count, dtype: int64
```

3. Exploratory data analysis and visualizations

The plots cover univariate distributions, class balance, bivariate relationships, multivariate correlation, and outlier inspection. For each plot, the notebook prints an interpretation prompt; use the displayed pattern and actual counts to write the final observation.

```
# Visualization 1: target class distribution
plt.figure(figsize=(8,5))
order = df[target_col].value_counts().index
sns.countplot(data=df, x=target_col, order=order)
plt.title("Distribution of Student Academic Outcomes")
plt.xlabel("Academic outcome"); plt.ylabel("Number of students")
plt.xticks(rotation=15); plt.tight_layout(); plt.show()
print("Interpretation: Compare the number of students in each outcome
class; note any class imbalance.")
```



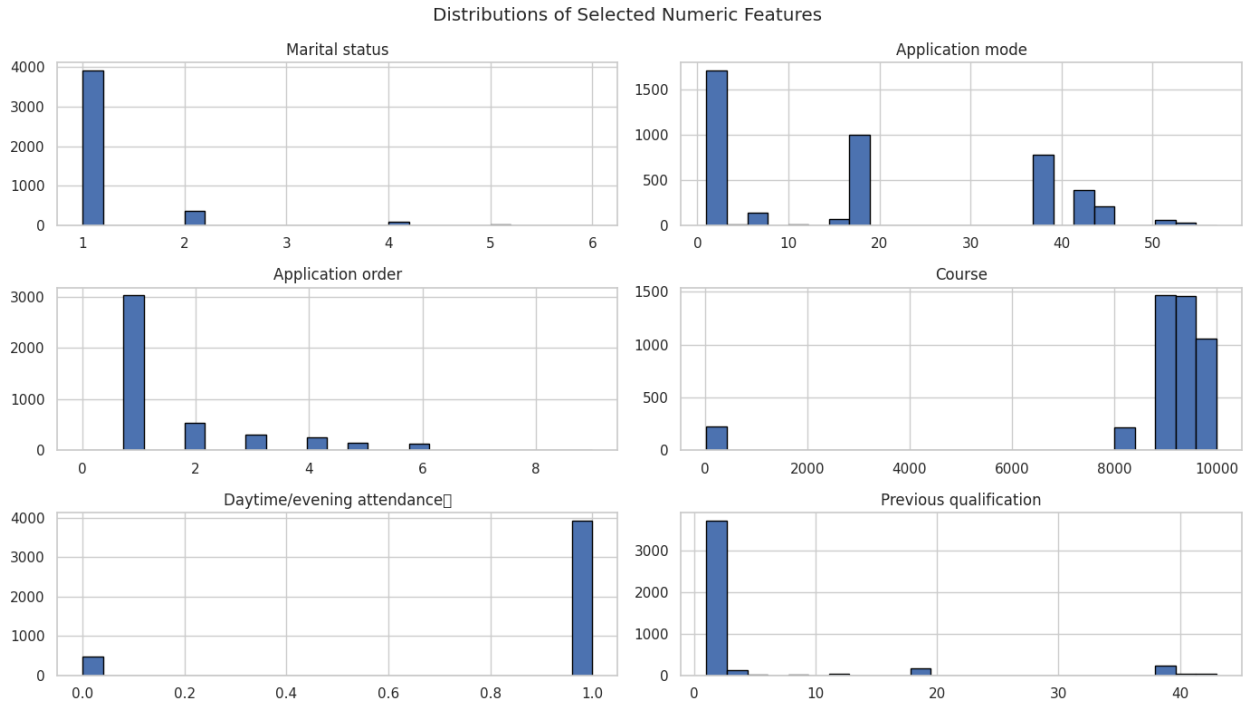

```
Interpretation: Compare the number of students in each outcome class;
note any class imbalance.
```

```
# Identify numeric and categorical columns
feature_cols = [c for c in df.columns if c != target_col]
numeric_cols =
df[feature_cols].select_dtypes(include=np.number).columns.tolist()
categorical_cols = [c for c in feature_cols if c not in numeric_cols]
```

```
print("Numeric features:", len(numeric_cols), "| Categorical
features:", len(categorical_cols))
```

```
Numeric features: 36 | Categorical features: 0
```

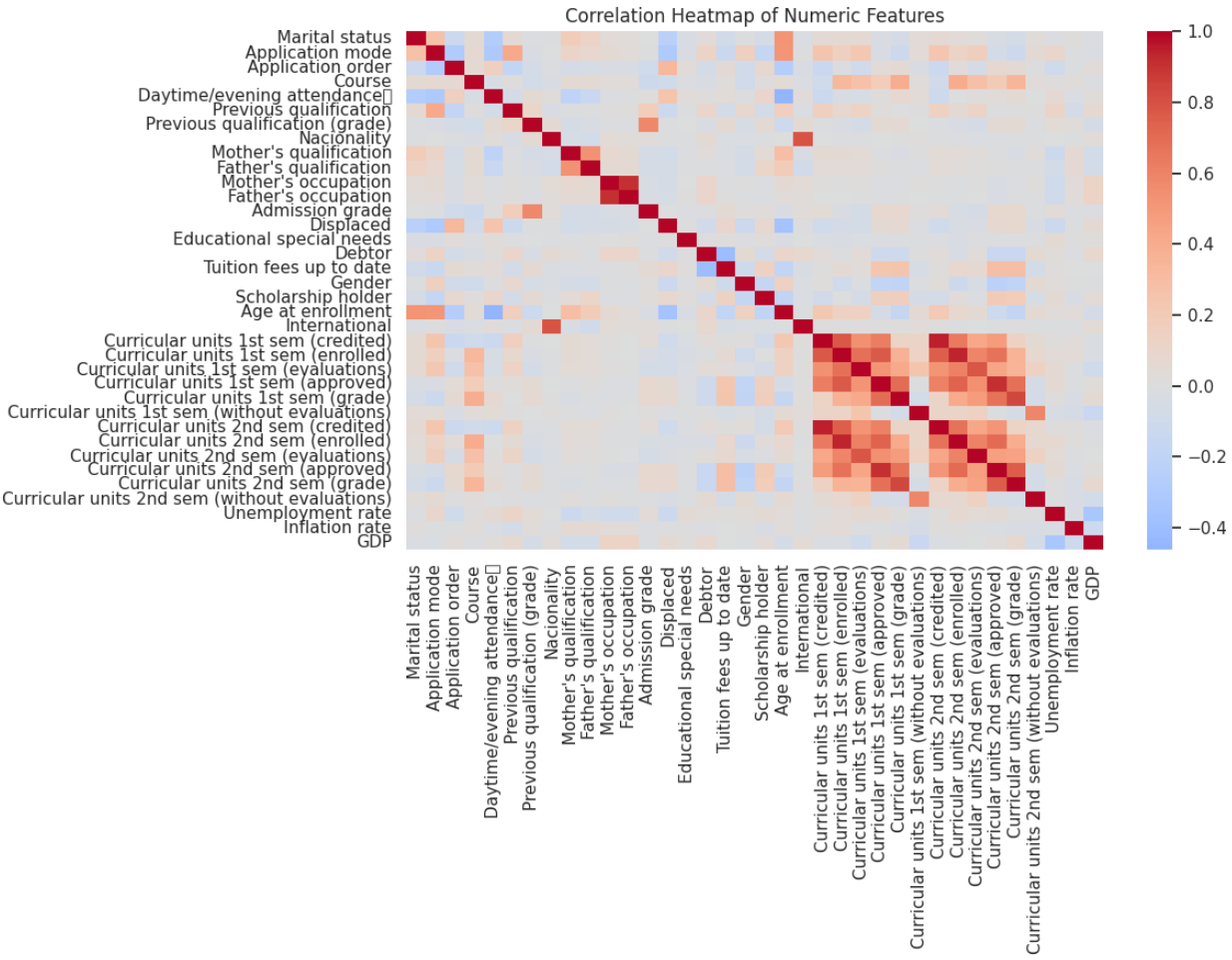
```
# Visualization 2: distributions of selected numeric features
selected_num = numeric_cols[:6]
if selected_num:
    df[selected_num].hist(figsize=(14,8), bins=25, edgecolor="black")
    plt.suptitle("Distributions of Selected Numeric Features")
    plt.tight_layout(); plt.show()
    print("Interpretation: Identify skewness, spread, and possible
unusual values in the selected features.")
else:
    print("No numeric columns available for histograms.")
```




```
Interpretation: Identify skewness, spread, and possible unusual values
in the selected features.
```

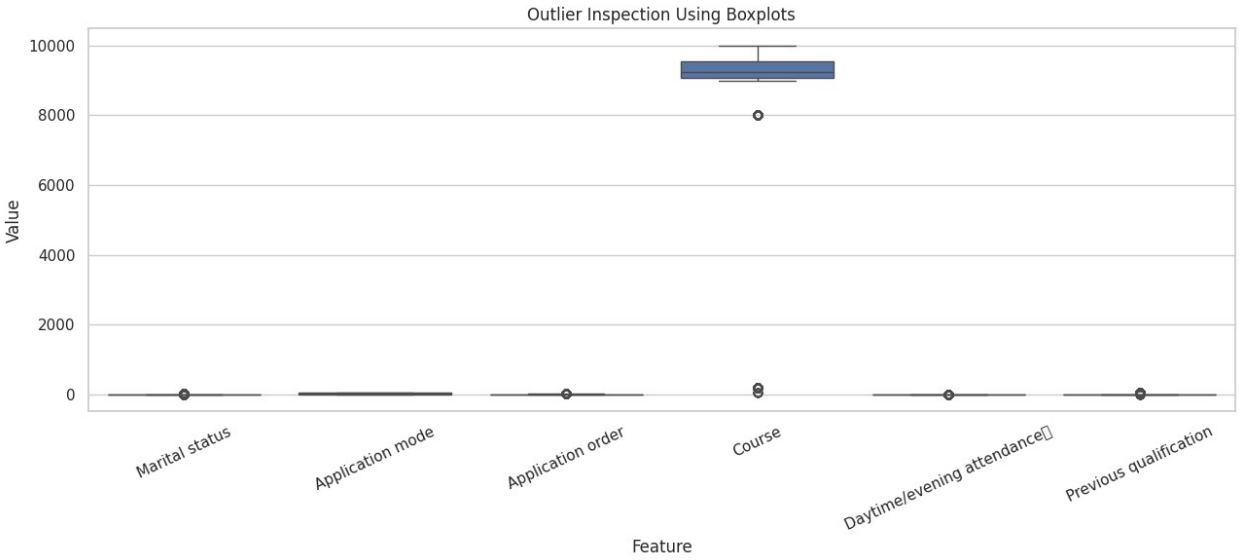
```
# Visualization 3: correlation heatmap for numeric features
if len(numeric_cols) >= 2:
    corr = df[numeric_cols].corr()
    plt.figure(figsize=(12,9))
    sns.heatmap(corr, cmap="coolwarm", center=0, annot=False)
    plt.title("Correlation Heatmap of Numeric Features")
    plt.tight_layout(); plt.show()
    print("Interpretation: Look for strongly positive or negative
correlations; correlation does not prove causation.")
```

```
else:
    print("Not enough numeric columns for correlation analysis.")
```



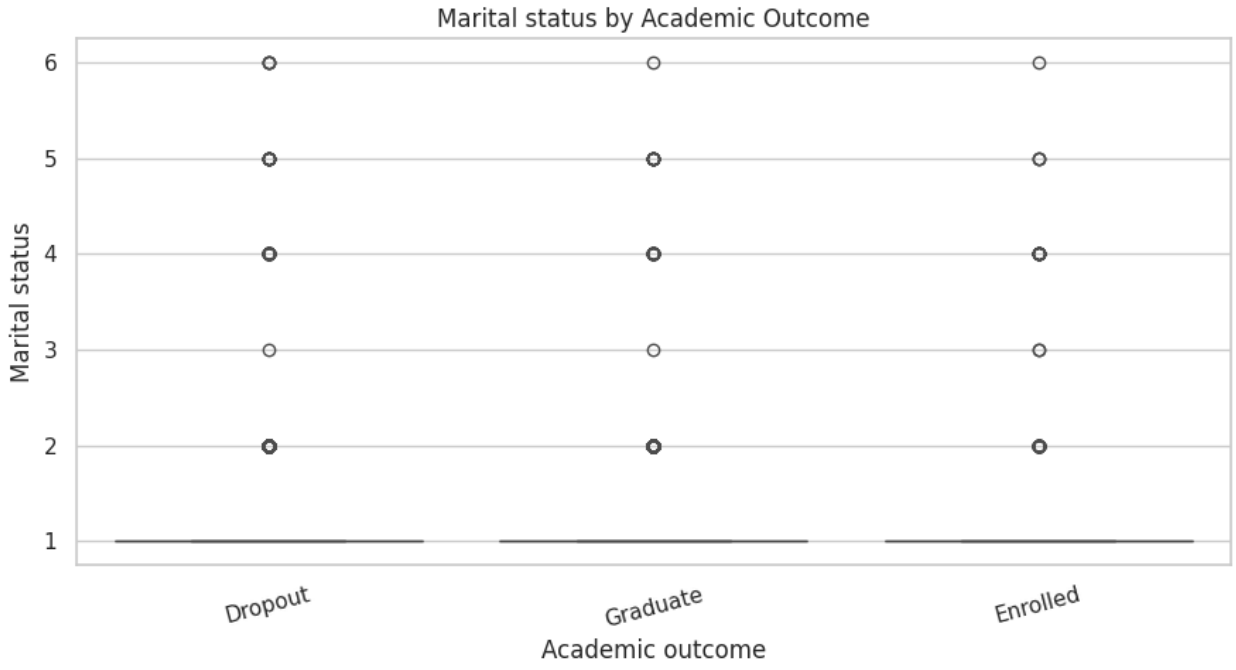

```
Interpretation: Look for strongly positive or negative correlations;
correlation does not prove causation.

# Visualization 4: boxplots for selected numeric features
if selected_num:
    long_df = df[selected_num].melt(var_name="Feature",
value_name="Value")
    plt.figure(figsize=(13,6))
    sns.boxplot(data=long_df, x="Feature", y="Value")
    plt.title("Outlier Inspection Using Boxplots")
    plt.xticks(rotation=25); plt.tight_layout(); plt.show()
    print("Interpretation: Points beyond whiskers may be potential
outliers; investigate before removing them.")
```




```
Interpretation: Points beyond whiskers may be potential outliers;
investigate before removing them.

# Visualization 5: feature relationship to outcome
# Use a numeric feature with variation; plot outcome-wise
distributions.
if numeric_cols:
    chosen = numeric_cols[0]
    plt.figure(figsize=(9,5))
    sns.boxplot(data=df, x=target_col, y=chosen)
    plt.title(f"{chosen} by Academic Outcome")
    plt.xlabel("Academic outcome"); plt.ylabel(chosen)
    plt.xticks(rotation=15); plt.tight_layout(); plt.show()
    print(f"Interpretation: Compare the distribution of {chosen}
across outcome groups; differences are associations, not proof of
cause.")
```




```
Interpretation: Compare the distribution of Marital status across
outcome groups; differences are associations, not proof of cause.

# Optional categorical frequency plot
if categorical_cols:
    c = categorical_cols[0]
    top = df[c].astype(str).value_counts().head(12).index
    plt.figure(figsize=(10,5))
    sns.countplot(data=df[df[c].astype(str).isin(top)], y=c,
order=top)
    plt.title(f"Most Frequent Categories: {c}")
    plt.tight_layout(); plt.show()
    print("Interpretation: Describe the most common categories and
whether the distribution is balanced.")
```

4. Feature analysis and train/validation/test split

Split data into training, validation, and test sets using stratification. The validation set is used for basic model selection; the test set is reserved for final evaluation.

```
X = df[feature_cols].copy()
y = df[target_col].copy()

# 70% train, 15% validation, 15% test
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.30, random_state=RANDOM_STATE, stratify=y)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=RANDOM_STATE,
```

```
stratify=y_temp)

num_pipe = Pipeline([("imputer", SimpleImputer(strategy="median")),
                     ("scaler", StandardScaler())])
cat_pipe = Pipeline([("imputer",
SimpleImputer(strategy="most_frequent")),
                     ("onehot",
OneHotEncoder(handle_unknown="ignore"))])
preprocessor = ColumnTransformer([
    ("num", num_pipe, numeric_cols),
    ("cat", cat_pipe, categorical_cols)
])

X_train_p = preprocessor.fit_transform(X_train)
X_val_p = preprocessor.transform(X_val)
X_test_p = preprocessor.transform(X_test)
print("Train:", X_train.shape, "Validation:", X_val.shape, "Test:",
X_test.shape)
```

```
Train: (3096, 36) Validation: (664, 36) Test: (664, 36)
```

5. Deep learning model: Multilayer Perceptron

An MLP is a feed-forward neural network suitable for tabular classification. We compare a small number of hidden-layer configurations on the validation set. The scikit-learn MLP provides loss history but does not provide per-epoch validation accuracy history directly; the notebook therefore reports validation performance for each configuration and plots the training loss curve for the selected model.

```
# Fix: convert preprocessed data into dense numeric arrays
X_train_fixed = np.asarray(X_train_p.toarray()
                           if hasattr(X_train_p, "toarray")
                           else X_train_p, dtype=np.float64)

X_val_fixed = np.asarray(X_val_p.toarray()
                         if hasattr(X_val_p, "toarray")
                         else X_val_p, dtype=np.float64)

X_test_fixed = np.asarray(X_test_p.toarray()
                          if hasattr(X_test_p, "toarray")
                          else X_test_p, dtype=np.float64)

configs = [
    {"hidden_layer_sizes": (64, 32), "alpha": 0.0001},
    {"hidden_layer_sizes": (100, 50), "alpha": 0.001},
]

results = []
models = []
```

```
for cfg in configs:
    model = MLPClassifier(
        **cfg,
        max_iter=150,
        early_stopping=False,
        random_state=42
    )

    model.fit(X_train_fixed, y_train)

    val_pred = model.predict(X_val_fixed)
    val_acc = accuracy_score(y_val, val_pred)

    results.append({
        **cfg,
        "validation_accuracy": val_acc,
        "iterations": model.n_iter_
    })

    models.append(model)

results_df = pd.DataFrame(results).sort_values(
    "validation_accuracy", ascending=False
)

display(results_df)

best_idx = results_df.index[0]
best_model = models[int(best_idx)]

print("Selected model:", configs[int(best_idx)])
```


```
{"summary":"{\n  \"name\": \"results_df\",\n  \"rows\": 2,\n
\"fields\": [\n    {\n      \"column\": \"hidden_layer_sizes\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n          [\n
100,\n            50\n          ],\n          [\n            64,\n
32\n          ]\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"alpha\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.0006363961030678928,\n        \"min\": 0.0001,\n
\"max\": 0.001,\n        \"num_unique_values\": 2,\n
\"samples\": [\n          0.001,\n          0.0001\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"validation_accuracy\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.015973797767768357,\n        \"min\": 0.6792168674698795,\n
\"max\": 0.7018072289156626,\n        \"num_unique_values\": 2,\n
\"samples\": [\n          0.6792168674698795,\n
```

```
0.7018072289156626\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"iterations\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0,\n        \"min\": 150,\n
\"max\": 150,\n        \"num_unique_values\": 1,\n        \"samples\":
[\n          150\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"results_df"}
```

```
Selected model: {'hidden_layer_sizes': (64, 32), 'alpha': 0.0001}
```

```
# Model evaluation on the untouched test set
test_pred = best_model.predict(X_test_p)
test_acc = accuracy_score(y_test, test_pred)
print(f"Test accuracy: {test_acc:.4f}")
print("\nClassification report:")
print(classification_report(y_test, test_pred, zero_division=0))
```

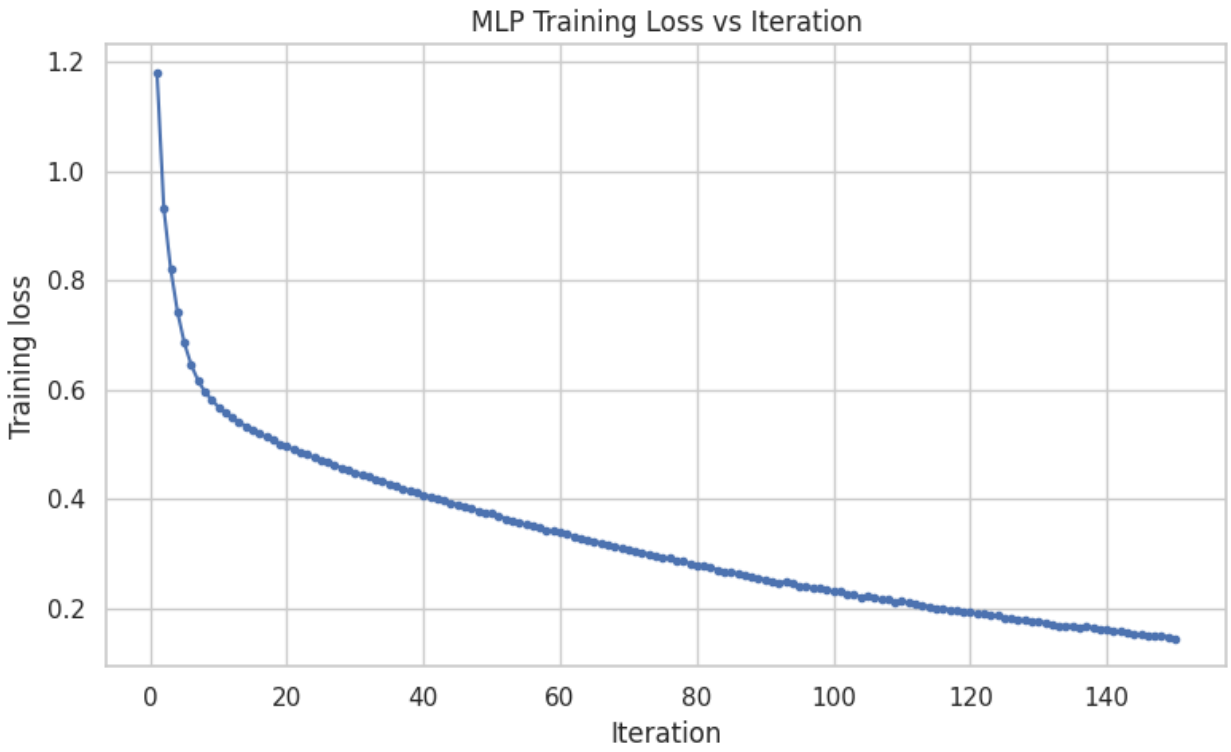
```
Test accuracy: 0.7364

Classification report:
              precision    recall  f1-score   support

     Dropout       0.77      0.77      0.77       214
    Enrolled       0.41      0.39      0.40       119
    Graduate       0.83      0.84      0.83       331

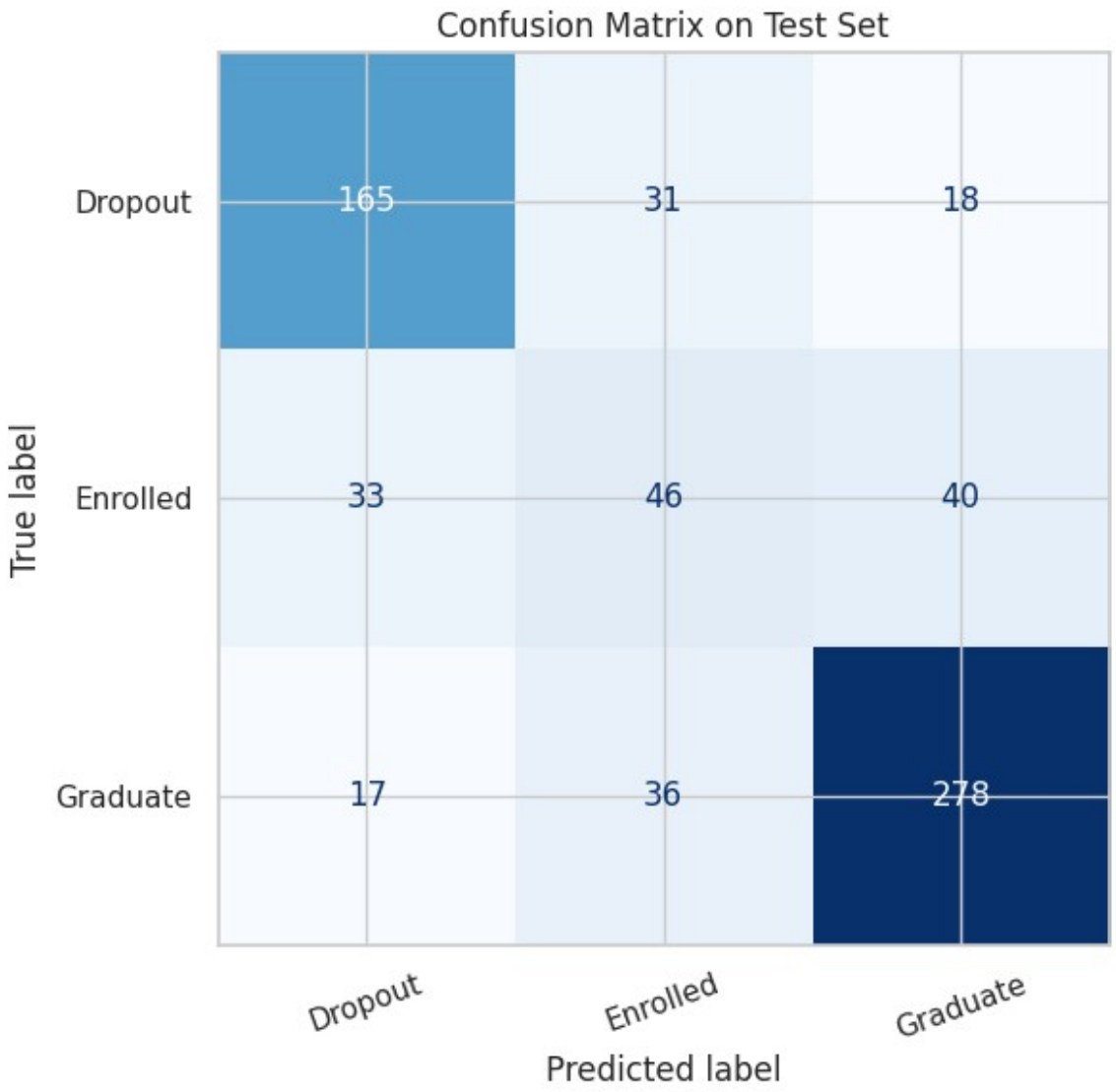
    accuracy                           0.74       664
   macro avg       0.67      0.67      0.67       664
weighted avg       0.73      0.74      0.73       664
```

```
# Evaluation visualization 1: loss vs iteration
plt.figure(figsize=(8,5))
plt.plot(range(1, len(best_model.loss_curve_) + 1),
best_model.loss_curve_, marker=".")
plt.title("MLP Training Loss vs Iteration")
plt.xlabel("Iteration"); plt.ylabel("Training loss")
plt.tight_layout(); plt.show()
print("Interpretation: A decreasing loss suggests learning; a plateau
may indicate convergence.")
```




```
Interpretation: A decreasing loss suggests learning; a plateau may
indicate convergence.

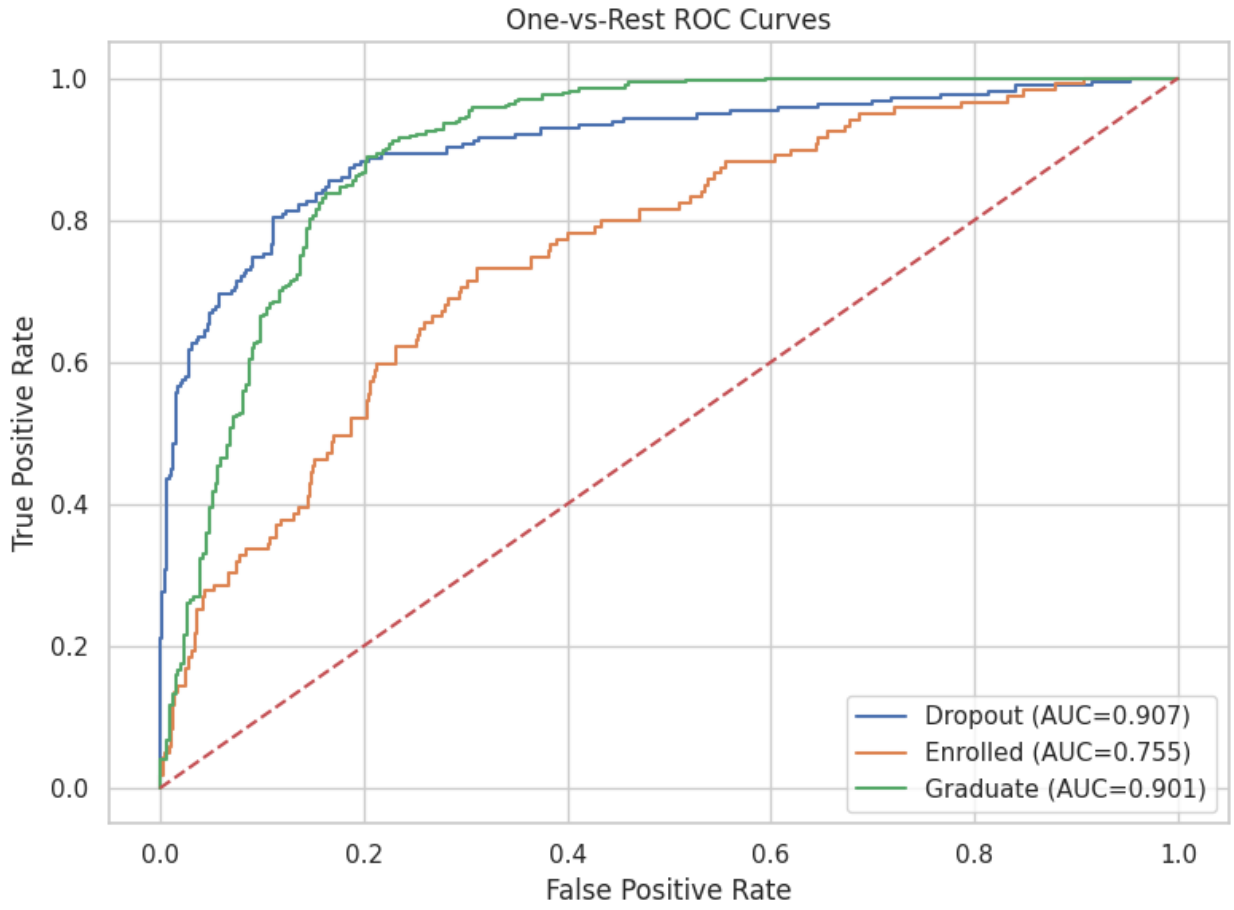
# Evaluation visualization 2: confusion matrix
labels = best_model.classes_
cm = confusion_matrix(y_test, test_pred, labels=labels)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=labels)
fig, ax = plt.subplots(figsize=(7,6))
disp.plot(ax=ax, cmap="Blues", xticks_rotation=20, colorbar=False)
plt.title("Confusion Matrix on Test Set")
plt.tight_layout(); plt.show()
print("Interpretation: Diagonal cells are correct predictions; off-
diagonal cells show misclassifications.")
```




```
Interpretation: Diagonal cells are correct predictions; off-diagonal
cells show misclassifications.

# Evaluation visualization 3: one-vs-rest ROC curves (if probability
scores are available)
from sklearn.preprocessing import label_binarize
proba = best_model.predict_proba(X_test_p)
classes = best_model.classes_
y_test_bin = label_binarize(y_test, classes=classes)
plt.figure(figsize=(8,6))
roc_results = []
for i, cls in enumerate(classes):
    if y_test_bin[:, i].min() == y_test_bin[:, i].max():
        continue
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], proba[:, i])
    roc_auc = auc(fpr, tpr)
```

```
    roc_results.append((cls, roc_auc))
    plt.plot(fpr, tpr, label=f"{cls} (AUC={roc_auc:.3f})")
plt.plot([0,1], [0,1], linestyle="--")
plt.xlabel("False Positive Rate"); plt.ylabel("True Positive Rate")
plt.title("One-vs-Rest ROC Curves")
plt.legend(); plt.tight_layout(); plt.show()
print("One-vs-rest AUC:", roc_results)
```

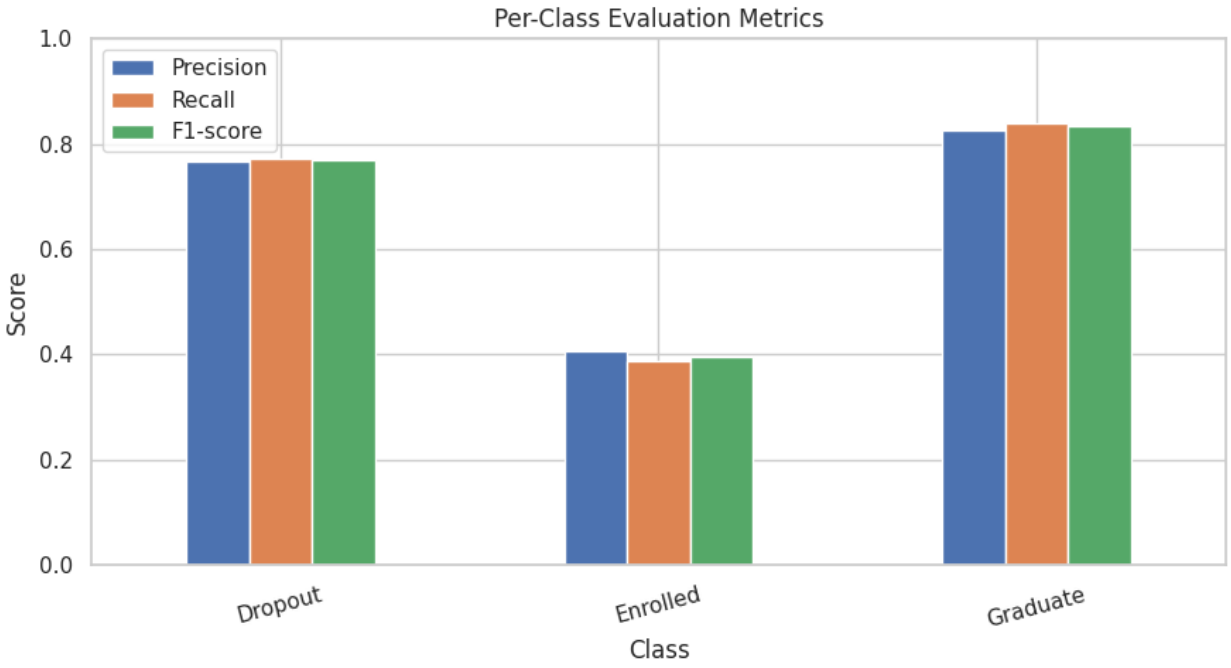



```
One-vs-rest AUC: [(np.str_('Dropout'),
np.float64(0.9069158878504673)), (np.str_('Enrolled'),
np.float64(0.754544753681289)), (np.str_('Graduate'),
np.float64(0.9009825535505294))]

# Evaluation visualization 4: per-class precision, recall, and F1
from sklearn.metrics import precision_recall_fscore_support
p, r, f1, support = precision_recall_fscore_support(y_test, test_pred,
labels=labels, zero_division=0)
metrics_df = pd.DataFrame({"Class": labels, "Precision": p, "Recall":
r, "F1-score": f1})
display(metrics_df)
metrics_df.set_index("Class").plot(kind="bar", figsize=(9,5),
```

```
ylim=(0,1))
plt.title("Per-Class Evaluation Metrics")
plt.ylabel("Score"); plt.xticks(rotation=15); plt.tight_layout();
plt.show()
```


```
{"summary":"{\n  \"name\": \"metrics_df\",\n  \"rows\": 3,\n
\"fields\": [\n    {\n      \"column\": \"Class\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 3,\n        \"samples\": [\n
\"Dropout\",\n          \"Enrolled\",\n          \"Graduate\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Precision\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.22734218279640295,\n        \"min\": 0.40707964601769914,\n
\"max\": 0.8273809523809523,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.7674418604651163,\n
0.40707964601769914,\n          0.8273809523809523\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Recall\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
0.24428924626498702,\n        \"min\": 0.3865546218487395,\n
\"max\": 0.8398791540785498,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.7710280373831776,\n
0.3865546218487395,\n          0.8398791540785498\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"F1-score\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
0.2359475313533487,\n        \"min\": 0.39655172413793105,\n
\"max\": 0.8335832083958021,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.7692307692307693,\n
0.39655172413793105,\n          0.8335832083958021\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n  ]\n}","type":"dataframe","variable_name":"metrics_df"}
```



6. Conclusion and future scope

Complete this section after running the notebook. Report the actual dataset dimensions, class distribution, missing/duplicate counts, the most notable plot findings, selected MLP configuration, test accuracy, and class-wise results. Do not claim a score before the notebook produces it.

Future scope

- Compare MLP with tree-based baselines and other neural-network configurations.
- Use cross-validation and systematic hyperparameter search.
- Assess fairness and the effect of socioeconomic variables carefully.
- Collect updated data and validate the model on a different cohort.
- Treat predictions as decision support, not as a substitute for academic counselling.

7. Seven stages of visualization

Use these stages explicitly in the report:

1. **Acquire:** obtain the dataset from UCI.
2. **Parse:** inspect columns, data types, and target labels.
3. **Filter:** remove duplicate rows and select relevant features.
4. **Mine:** calculate descriptive statistics, distributions, and correlations.
5. **Represent:** choose count plots, histograms, heatmaps, boxplots, and outcome-wise comparisons.
6. **Refine:** add titles, axis labels, readable ticks, legends, and suitable scales.
7. **Interact:** use notebook controls such as displayed tables and rerunnable cells to inspect different features; optional interactive charts can be added with Plotly.

References

1. Realinho, V., Machado, J., Baptista, L., & Martins, M. V. (2022). *Predict students' dropout and academic success* [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5MC89
2. Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.
3. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95.
4. Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software, 6*(60), 3021.
5. McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.